

High-Level Design (HLD) – Functional & Non-Functional Requirements

System Design starts by understanding **what the system should do** and **how well it should do it**.

These are divided into:

1. **Functional Requirements** → Features / behavior of the system
2. **Non-Functional Requirements** → Quality attributes / constraints / system characteristics

1. Functional Requirements

Functional requirements define the **inputs, outputs, and operations** of a system.

They answer:

- What services should the system provide?
- What actions can users perform?
- What APIs are needed?
- What business logic must happen?

These are often represented using:

- User stories
- Use cases
- API endpoints
- Workflow diagrams
- Database operations

Examples of Functional Requirements

E-commerce System

- User signup/login
- Search products
- Add to cart
- Place order
- Make payment
- Track shipment
- Cancel order
- Return/refund

Social Media App

- Create post
- Like/comment/share
- Follow users
- Chat system
- Notifications
- Media upload

Food Delivery App

- Browse restaurants
- Place order
- Real-time tracking
- Rating system
- Coupon application

Example API Mapping

None

POST /signup

POST /login

GET /products

POST /cart/add

POST /order/create

GET /order/{id}

2. Non-Functional Requirements (Most Important in HLD)

These define **how the system performs under constraints**.

They determine architecture choices.

Key Non-Functional Requirements

1. Scalability

Definition:

Ability of a system to grow and handle increasing traffic efficiently.

Types:

Vertical Scaling (Scale Up)

Increase server power:

- More CPU
- More RAM
- Better SSD

Horizontal Scaling (Scale Out)

Add more servers behind load balancer.

Example:

Instagram growing from 1M to 100M users requires horizontal scaling.

Techniques:

- Load balancers
- Stateless services
- Sharding
- Caching
- CDN
- Auto-scaling groups

2. Availability

Definition:

Percentage of time system remains operational.

Formula:

None

$\text{Availability} = \text{Uptime} / \text{Total Time}$

Examples:

Availability	Downtime / Year
99%	3.65 days
99.9%	8.76 hrs
99.99%	52 min
99.999%	5 min

Techniques:

- Redundant servers
- Failover systems
- Multi-region deployment
- Health checks
- Auto recovery

3. Performance / Latency / Throughput

Latency

Time taken for request → response.

Example:

- Search response = 120ms

P99 Latency

99% requests finish below a time limit.

Example:

- P99 = 300ms

Means only 1% slower than 300ms.

Throughput

Requests processed per second.

Example:

- 20,000 requests/sec

Techniques to Improve Performance

- Caching
- DB indexing
- Async processing
- CDN
- Efficient queries
- Load balancing

4. Fault Tolerance

Definition:

System continues working even when components fail.

Examples:

- Server crash
- DB node failure
- Network outage

Techniques:

- Replication
- Retries
- Circuit breaker
- Backup systems
- Queue buffering
- Graceful degradation

Example:

If recommendation engine fails → app still loads products.

5. Security

Definition:

Protect system from unauthorized access.

Includes:

- Authentication
- Authorization
- Encryption
- API security
- Rate limiting
- WAF
- DDoS protection

Techniques:

- OAuth / JWT
- TLS/HTTPS
- RBAC
- Input validation
- Secrets management

6. Privacy

Definition:

Protect user personal data (PII).

PII includes:

- Name
- Email
- Phone
- Aadhaar / SSN
- Address
- Payment info

Techniques:

- Encryption at rest
- Access control
- Data masking
- GDPR compliance
- Audit logs

7. Accuracy

Definition:

Correctness of data/results.

Examples:

- Bank balance must be exact
- Inventory count should be correct
- Analytics dashboard can tolerate slight delay

Tradeoff:

High accuracy often increases cost and latency.

8. Consistency

Definition:

All replicas show same data.

Example:

After money transfer balance must match everywhere.

Types:

Strong Consistency

All users see latest data immediately.

Eventual Consistency

Temporary mismatch allowed.

Eventually synced.

Examples:

- Banking → Strong consistency
- Social media likes count → Eventual consistency

9. Cost

Definition:

Infrastructure + engineering + maintenance cost.

Includes:

- Servers
- Storage
- CDN
- Monitoring tools
- Team effort

Example Tradeoff:

Using Redis improves speed but increases cost.

10. Complexity / Maintainability / Debuggability / Testability

Complexity

How hard system is to understand.

Maintainability

How easy future changes are.

Debuggability

How fast issues can be diagnosed.

Testability

How easy system can be tested.

Improve By:

- Modular architecture
- Microservices
- Clear APIs
- Logging
- Monitoring
- Documentation
- Automated tests

Important Tradeoffs in HLD

System design is all about tradeoffs.

Improve	Cost
Scalability	Complexity

Availability	More infra
Security	Latency
Consistency	Lower performance
Performance	Higher cost
Accuracy	More compute

Interview Tip

Always ask:

Functional:

- Who are users?
- What features?
- Read heavy or write heavy?
- Real-time needed?

Non-Functional:

- QPS?
- Latency target?
- Expected DAU/MAU?
- Availability target?
- Data size?
- Consistency needed?
- Budget constraints?

Real Example

WhatsApp Priorities:

- Availability high
- Low latency
- Scalability huge
- Eventual consistency acceptable for read receipts

Banking System Priorities:

- Strong consistency
- Security
- Accuracy

- Auditability

Golden Rule

Functional requirements decide **WHAT to build**

Non-functional requirements decide **HOW to build**